

TRAINING COMPLETION REPORT

AI-Ready Python Full Stack Engineering Program
Target: BE / B.Tech Artificial Intelligence & Machine Learning
(AI/ML)

SUBMITTED IN FULFILLMENT OF THE TRAINING PROGRAM

Submitted To: The Head of Department
Department of Artificial Intelligence &
Machine Learning
Alvas Institute of Engineering &
Technology, Moodbidri

Submitted By: Techademy Learning Solutions Private
Limited
Bengaluru, India

Trainer Details: Mr. Sriram Murugesan
Lead Technical Trainer

Program Duration: 15 Days / 90 Contact Hours
(09:00 AM – 05:00 PM Daily)

Submission Date: July 16, 2026

Executive Summary

The AI-Ready Python Full Stack Engineering program was delivered as a 15-day intensive, outcome-oriented training initiative, designed to prepare final-year BE/B.Tech Artificial Intelligence & Machine Learning (AI/ML) students at Alvas Institute of Engineering & Technology for competitive roles in modern product startups and established technology companies. Conducted from 09:00 AM to 05:00 PM daily, the 90-hour curriculum balanced concept sessions (25%) with structured, live-coded guided laboratory exercises and independent practice (75%).

Adhering to a modern Agile model, the program took students through successive phases: building robust, asynchronous backends with FastAPI and PostgreSQL; developing components, hooks, and typed layouts with React 18 and TypeScript; and introducing advanced AI application concepts including Large Language Models (LLMs), semantic search, custom prompt engineering, conversation memory, and Retrieval-Augmented Generation (RAG) with vector databases. In the final phase, students learned containerization using Docker, orchestration via Docker Compose, and cloud deployment strategies on Amazon Web Services (AWS) EC2/S3, as well as Vercel and Render.

Rather than building a single shared application, students were organized into project teams to design and build original full-stack AI applications based on their own unique ideas. These projects successfully integrated authentication, database persistence, custom LangChain components, local embeddings, vector similarity search, containerized multi-service setups, and public deployment on AWS, Vercel, and Render, culminating in final Capstone Presentations evaluated by a panel of their own faculty.

Program Objectives

The primary training objectives of this program were to:

- Transition students from synchronous paradigms (Django/Flask) to high-concurrency asynchronous API development with FastAPI, Pydantic, and dependency injection.
- Equip students to model, link, migrate, and query relational databases using PostgreSQL and SQLAlchemy's asynchronous ORM layer.
- Enable the construction of fully typed, component-based frontends in React 18 + TypeScript using advanced hooks, structured state management, and unified Axios service clients.
- Implement industry-standard security practices, including JWT authentication, password hashing with bcrypt, role-based access control (RBAC), and persistent state authentication in React.

- Introduce generative AI integration using LLMs (Groq, OpenAI), prompt templates, and custom conversational workflows.
- Implement semantic search, embeddings, and Retrieval-Augmented Generation (RAG) using FastEmbed and Qdrant Vector Database.
- Build containerized local environments for multi-service architectures using Dockerfiles and Docker Compose.
- Train students on cloud deployment processes, setting up Ubuntu-based AWS EC2 virtual machines, managing security groups, and configuring environment secrets.
- Enable deployment using AWS as well as developer-friendly platforms such as Vercel and Render.
- Promote collaborative software development practices using Git, GitHub, clean commit practices, and structured documentation.

Technology Stack Covered

The curriculum was carefully structured to align with tools used in contemporary AI-first startups and mid-to-large product organizations. The complete list of libraries, frameworks, and database services includes:

Layer / Domain	Technologies, Libraries & Services
Backend Development	Python, FastAPI, Pydantic v2, SQLAlchemy Async ORM, Alembic migrations, Uvicorn.
Database / Persistence	PostgreSQL (hosted on Neon Database Server), SQL queries, relational database modeling.
Frontend Development	React 18, TypeScript, Axios, React Router, Global Vanilla CSS design system.
Artificial Intelligence & NLP	LangChain, Groq API (Llama 3.3 models), FastEmbed (local vectors), Qdrant Vector Database (Qdrant Cloud).
DevOps & Cloud Hosting	Docker, Docker Compose, AWS EC2, AWS S3, Vercel, Render, environment variables, GitHub Actions.

Day-wise Training Details

Day 1

Planned Syllabus: FastAPI Introduction, Architecture, Project Structure, Routers, Models, Schemas, REST APIs.

Topics Delivered: Introduction to backend engineering paradigms, asynchronous processing in Python, FastAPI project layout, Uvicorn ASGI server, APIRouter routing, request handlers, path parameters, query parameters, Pydantic model schemas, and automatic API documentation (Swagger UI/Redoc).

Hands-on Activities: Setting up local workspace, implementing basic GET/POST endpoints, creating a Pydantic schema validation layer for 'Company' and 'Job' structures, writing mock CRUD endpoints.

Learning Outcomes: Students acquired a clear understanding of the request-response lifecycle, parameter validation using Pydantic, and API documentation standard flows.

Day 2

Planned Syllabus: PostgreSQL, SQLAlchemy, Database Connectivity, Relationships, Alembic.

Topics Delivered: Relational database design, Neon cloud PostgreSQL server provisioning, SQLAlchemy asynchronous engine setup, AsyncSession transaction contexts, declarative mapping models, relational foreign key structures, and async DB query implementations.

Hands-on Activities: Configuring database connection variables, writing async models for User, Company, and Job entities, linking tables, implementing CRUD helper functions, and exposing database operations through FastAPI endpoint handlers.

Learning Outcomes: Students mastered the mapping of relational schemas to Python classes, async database connections, session management, and relational mapping logic.

Day 3

Planned Syllabus: React, TypeScript, Components, Hooks.

Topics Delivered: React 18 component-driven paradigm, TypeScript type safety in frontend interfaces, prop declarations, component lifecycle states, React standard hooks (useState, useEffect, useContext), and building responsive reusable layouts.

Hands-on Activities: Initializing a Vite + React + TypeScript workspace, drafting a global CSS design system, designing card-based dashboards, component creation for job listings, and setting up routing layouts.

Learning Outcomes: Students understood React rendering mechanisms, TypeScript compiler type-guards, components props declarations, and basic hooks lifecycle operations.

Day 4

Planned Syllabus: React Integration, API Consumption, Axios, Error Handling.

Topics Delivered: Axios service clients setup, standard async/await API ingestion patterns, loading indicators, error boundaries, page-to-page routing with React Router, and connecting local storage tokens to state variables.

Hands-on Activities: Implementing Axios client service methods, linking FastAPI REST endpoints to React dashboard modules, writing state routines for search filters, and handling server errors.

Learning Outcomes: Students successfully bound front-end views to real-time back-end databases, enabling working end-to-end CRUD operations on a single local host setup.

Day 5

Planned Syllabus: JWT Authentication, OAuth2, Protected Routes.

Topics Delivered: Security principles, JWT mechanics (header, payload, signature), hashing passwords using bcrypt, token duration configurations, OAuth2 Password Flow, and securing FastAPI routers with dependency injection.

Hands-on Activities: Writing registration and login endpoints, generating JWT tokens, creating a 'current_user' dependency in FastAPI, testing authenticated routes via Swagger UI, and establishing state management to store tokens.

Learning Outcomes: Students understood state validation, secure password storage, token generation pipelines, and basic concepts of resource authorization.

Day 6

Planned Syllabus: JWT Authentication, OAuth2 Password Flow, Protected APIs, Role-Based Access Control, Full Stack Authentication.

Topics Delivered: Deep dive into multi-role credentials, JWT verification middleware, role validation routines, HTTP bearer interceptors in Axios, setting headers, and client-side page permission checks.

Hands-on Activities: Implementing role flags ('recruiter' vs 'candidate') in DB models, coding API route guards, creating login forms in React, caching tokens in localStorage, and writing interceptors to handle 401 Unauthorized API responses.

Learning Outcomes: Students successfully built a complete authenticated full-stack application, ensuring secure page redirection and restricted database access based on user roles.

Day 7

Planned Syllabus: LLM Fundamentals, Prompt Engineering, LangChain, AI Integration.

Topics Delivered: Large Language Models (LLMs) concepts, tokens, context windows, API parameters (temperature, top_p), system prompts vs human prompts, LangChain architecture, and prompt template runnables.

Hands-on Activities: Configuring Groq API connections (using Llama 3.3 model), building custom prompt templates, chaining inputs, and establishing text processing flows using LangChain's RunnableSequence.

Learning Outcomes: Students understood parameter configurations of LLMs, structuring prompts, and constructing simple LangChain sequences for text parsing.

Day 8

Planned Syllabus: AI Chatbot, LangChain Memory, Streaming Responses.

Topics Delivered: Conversational agents design, conversation history buffers, Session-based memory managers, RunnableWithMessageHistory, and Server-Sent Events (SSE) streaming API patterns.

Hands-on Activities: Building a chat API endpoint in FastAPI, mapping Session IDs to ChatMessageHistory, writing streaming text components in React, and creating an interactive typing-bubble interface.

Learning Outcomes: Students built an operational multi-turn AI chatbot that retains conversation context and streams completions back to the UI in real-time.

Day 9

Planned Syllabus: RAG, Embeddings, Vector Database, Semantic Search.

Topics Delivered: Retrieval-Augmented Generation (RAG) concepts, semantic search vs keyword search, vector representations, FastEmbed embedding generation, Qdrant Vector DB, collection schemas, and Cosine similarity metrics.

Hands-on Activities: Setting up a Qdrant Cloud cluster, writing a background ingestion script, converting raw job texts into 384-dimensional dense vectors, storing vector points with payloads, and running vector similarity queries.

Learning Outcomes: Students learned the mathematics of cosine similarity, dense vector stores, collection schemas, and embedding extraction procedures.

Day 10

Planned Syllabus: AI Feature Integration, Resume Analysis, Smart Job Matching.

Topics Delivered: Integrating RAG systems with API routers, parsing resumes, dynamic context building, merging semantic matches with SQL filters, and Sprint 2 performance review.

Hands-on Activities: Writing a resume analyser API that extracts skills and queries Qdrant for matching jobs, integrating the semantic query engine into the React search page, and reviewing complete AI services.

Learning Outcomes: Students successfully deployed a full RAG workflow, linking search queries, database facts, vector representations, and LLM text generation into unified services.

Day 11

Planned Syllabus: Docker Images, Containers, Dockerfile, Docker Compose, Environment Variables.

Topics Delivered: Docker containerization concepts, building images, layer caching, Dockerfile specifications, Docker Compose orchestrations, virtual networks, and container environment injection.

Hands-on Activities: Writing independent Dockerfiles for Python-FastAPI and React-TypeScript, writing a docker-compose.yml file linking the backend, frontend, and database services, and setting up local network bridges.

Learning Outcomes: Students understood the benefits of container isolation, multi-service network structures, environment file separations, and local production simulation.

Day 12

Planned Syllabus: AWS, Vercel, and Render Deployment, Cloud Hosting, Environment Variables, File Storage.

Topics Delivered: AWS EC2 virtual machines, security groups, inbound/outbound rules, SSH access keys, public IP routing, and AWS S3 bucket configuration for

object storage. Introduction to developer-friendly cloud platforms: Vercel for fast React frontend deployment, and Render for hosting FastAPI web services and backend APIs.

Hands-on Activities: Setting up an EC2 Linux server, configuring security policies, deploying backend services on Render and AWS, deploying frontends on Vercel and Render, and checking storage setups in S3.

Learning Outcomes: Students successfully understood cloud deployment choices, deploying backends and frontends across AWS, Vercel, and Render, and configuring environment secrets in production environments.

Day 13

Planned Syllabus: Capstone Completion, Code Review, System Design.

Topics Delivered: Project scope management, Git workflow audits, code quality criteria, error handling, rate limiting on AI endpoints, and basic software system design concepts.

Hands-on Activities: Dividing into project teams, allocating unique domains, fixing code lints, auditing git histories, drafting system architectures, and preparing deployment configurations for Vercel, Render, and AWS.

Learning Outcomes: Students learned to manage code deliverables under deadlines, handle exceptions, structure repositories, and configure production-ready hosting environments.

Day 14

Planned Syllabus: End-to-End Testing, Presentation Preparation, Project Validation.

Topics Delivered: Testing methodologies, user journey dry runs, security audits, database migration validation, and scalability configurations.

Hands-on Activities: Conducting systematic end-to-end QA tests, compiling final README documentations, designing architecture diagrams, running mock technical reviews, and testing multi-platform deployment on AWS, Vercel, and Render.

Learning Outcomes: Students gained confidence in validation workflows, performance auditing, multi-platform hosting verification, and explaining system architecture details.

Day 15

Planned Syllabus: Capstone Presentations, Technical Evaluation, Panel Q&A, Program Closure.

Topics Delivered: Technical communication, demo coordination, technical critique methodologies, and final project evaluation.

Hands-on Activities: Conducting capstone project presentations and live demonstrations. Students presented their fully functioning full-stack applications, showing backend APIs, React interfaces, AI chatbots, and vector search features. The evaluation and presentations were conducted by their own faculty panel, who asked technical and architectural questions.

Learning Outcomes: Students successfully demonstrated complete AI full-stack development skills, defended their engineering decisions before the college faculty panel, and received feedback for their portfolios.

Capstone Projects Execution

To ensure academic honesty, foster creativity, and build diverse portfolios, the final capstone phase did not use a single shared repository. Instead, students worked in teams to build unique, custom solutions. Each project team was evaluated on their ability to deliver a production-ready application containing:

- **FastAPI Backend:** Fully asynchronous routers, proper Pydantic validations, and error middleware.
- **React Frontend:** Fully typed TypeScript layouts, API service wrappers, and responsive UI components.
- **Relational Persistence:** PostgreSQL tables for profiles, records, and activities, managed with SQLAlchemy.
- **Role-Based Authentication:** Protected paths and forms, with login and authorization handled by JWT.
- **Generative AI Integration:** Custom chatbot systems with context-retaining history buffers, using LangChain and Groq LLMs.
- **Retrieval-Augmented Generation (RAG):** Local text chunk indexing and vector storage in Qdrant Vector Database to support semantic searches.
- **Docker Orchestration:** Containerized services structured using Dockerfile configurations and docker-compose.yml files.
- **Multi-Platform Cloud Deployment:** Live hosting of production services using AWS (EC2/S3) as well as developer-friendly platforms Vercel (for frontend hosting) and Render (for backend web service hosting).

Sample project domains successfully developed, containerized, and deployed by the student teams included: AI-Powered Job Matching Platforms, Resume Screening Assistants, Semantic Helpdesk Systems, and Career Prep Chatbots.

Assessment & Evaluation Methodology

Student progress was tracked through a continuous assessment model. The final evaluation was based on the following criteria:

- **Daily Labs & Git Commits (25%):** Timely pushes to individual GitHub repositories showing incremental commits and clean code formatting.
- **Sprint Reviews (25%):** Practical tests on API development and AI/Authentication setups.
- **Final Capstone Project (40%):** Evaluation of the application structure, functional features, container setup, and deployment on AWS, Vercel, and Render.
- **Technical Presentation & Q&A (10%):** The team's ability to explain their engineering decisions and answer technical questions from the faculty evaluation panel.

Trainer Remarks & Recommendations

The seventh-semester Artificial Intelligence & Machine Learning (AI/ML) students at Alvas Institute of Engineering & Technology demonstrated strong interest and capability throughout this intensive program. Key findings include:

- **Hands-on Engagement:** Students quickly picked up asynchronous programming in FastAPI and building typed React interfaces with TypeScript.
- **AI Concept Adoption:** The batch easily grasped the fundamentals of vector representations, semantic indexing in Qdrant, and building LangChain workflows.
- **Deployment Competency:** All student teams successfully deployed their containerized applications to live cloud environments, utilizing AWS, Vercel, and Render to host their backend APIs and frontend pages.
- **Faculty Evaluation:** The final presentations demonstrated high technical fluency. The projects were evaluated directly by the college's own faculty panel, who engaged in thorough discussions on architecture, scalability, and vector indexing.

Conclusion

The 15-day AI-Ready Python Full Stack Engineering program was successfully completed, meeting all the requirements set by Techademy Learning Solutions and Alvas Institute of Engineering & Technology. By developing, dockerizing, and deploying their own AI full-stack capstone projects on AWS, Vercel, and Render, students gained practical, industry-aligned software engineering skills. They are now well-prepared for roles like Associate Software Engineer, Full Stack Developer, and AI Application Developer.

Report Compiled and Submitted By:

Techademy Learning Solutions Private Limited

Lead Instructor: Mr. Sriram Murugesan